Date __/__/__
Page _____

System software and compiler design

<tokenname, attributename>

1) E = M * C ^ ** 2

Address	lexeme	type	line_no
1000	E	float	2
2000	M	float	2
3000	C	float	2

<id, 1000>
<assign>
<id, 2000>
<mulp>
<id, 3000>
<exp_op>
<num, ~~1000~~>

2) Si = S*t *R/100

<id, 1000>
<assign>
<id, 2000>
<mulp>
<id, 3000>
<mulp>
<id, 4000>
<divide>
<num, 100>

Date __/__/__
Page

Input Buffering (Imp)

The method of reading a block of characters (1K/4K or More bytes) from the disk in one read operation and storing in memory (normally in the form of array) for further processing and faster access is called Input buffering.

Strings and languages :-

*) An alphabet is any finite set of symbols

*) String over an alphabet is a finite sequence of symbols drawn from that alphabet.

s = sound

*) length of string $|s| = 5$

*) Empty string ϵ $|s| = 0$

*) language is any countable set of string over some fixed alphabet.

*) operation on languages.

operation	Defn and Notation
union of L & M	$L \cup M = \{s \mid s \text{ is in } L \text{ or } s \text{ is in } M\}$
concatenation of L & M	$LM = \{st \mid s \text{ is in } L \text{ and } t \text{ is in } M\}$
Kleene closure of L	$L^* = \bigcup_{t=0}^{\infty} L^i$
positive closure of L	$L^+ = \bigcup_{i=1}^{\infty} L^i$

? ⇒ * [0 or more occur.]

Date __/__/__
Page _____

Regular Expressions

$r \Rightarrow L(r)$
$s \Rightarrow L(s)$

letter = (letter_ / digit)*
(or)

Eg: ab
a_
a1

*) $(r) | (s) \Rightarrow L(r) \cup L(s)$

*) $(r)(s) = L(r).L(s)$
*) $(r)* = (L(r))^*$
*) $r = L(r)$

*) ? is also a [0 or more occurrences]

digit → [0 - 9]
digits → [0-9]*
[digit]*

3.462^{+432}

number = digits (.) (digit)? (E (+-)? digits)?

) ws ⇒ (blank | tab | newline)

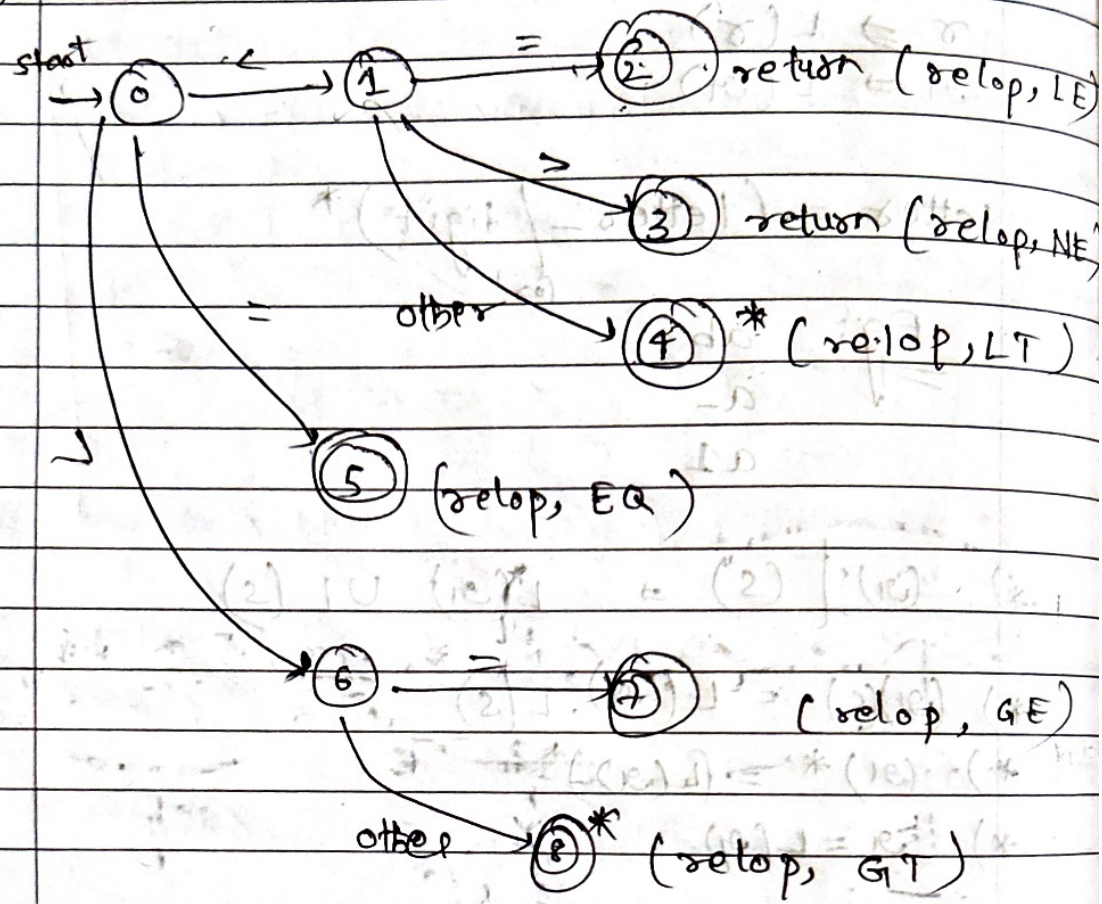
then

phases

< , <= , <> , = , >

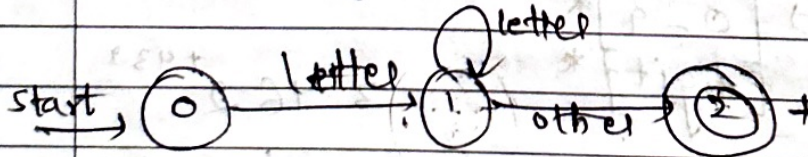
Date
Page

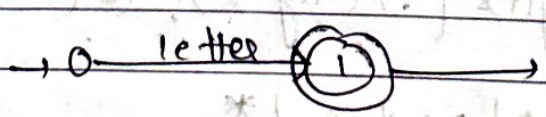
1)



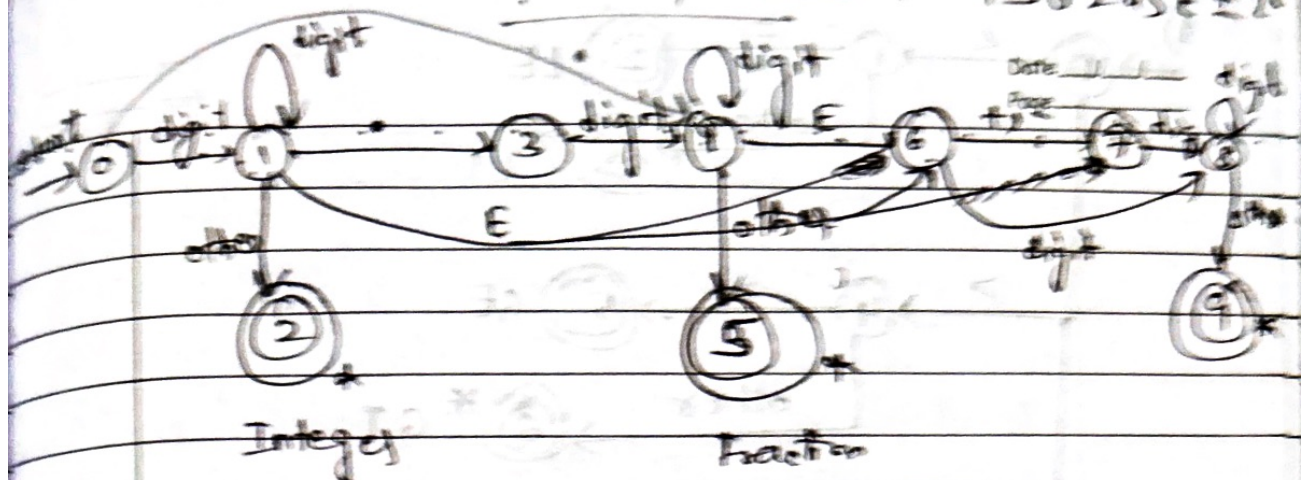
2).

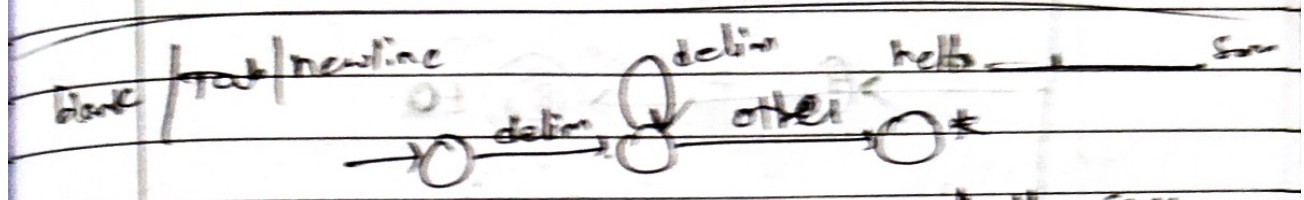
Hello.1

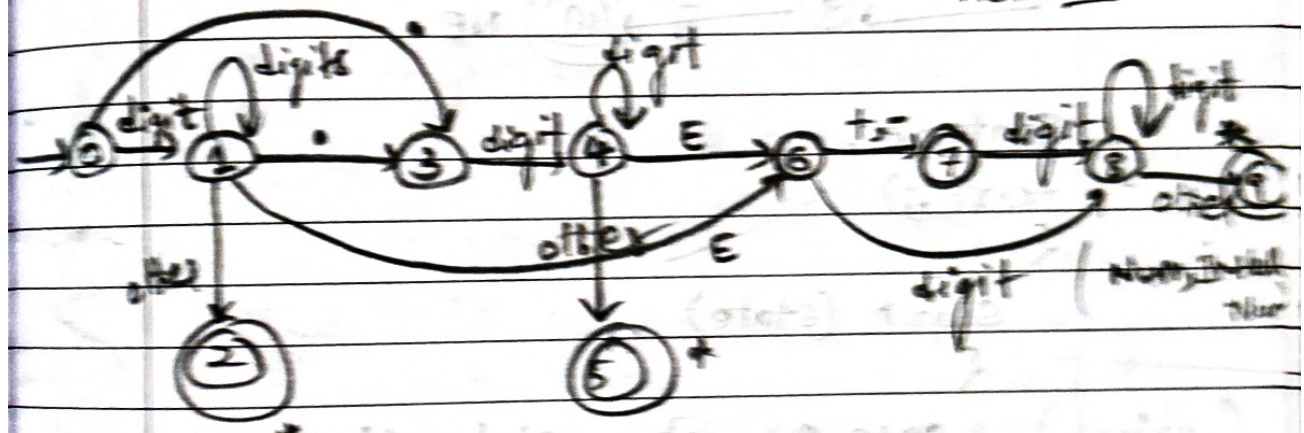




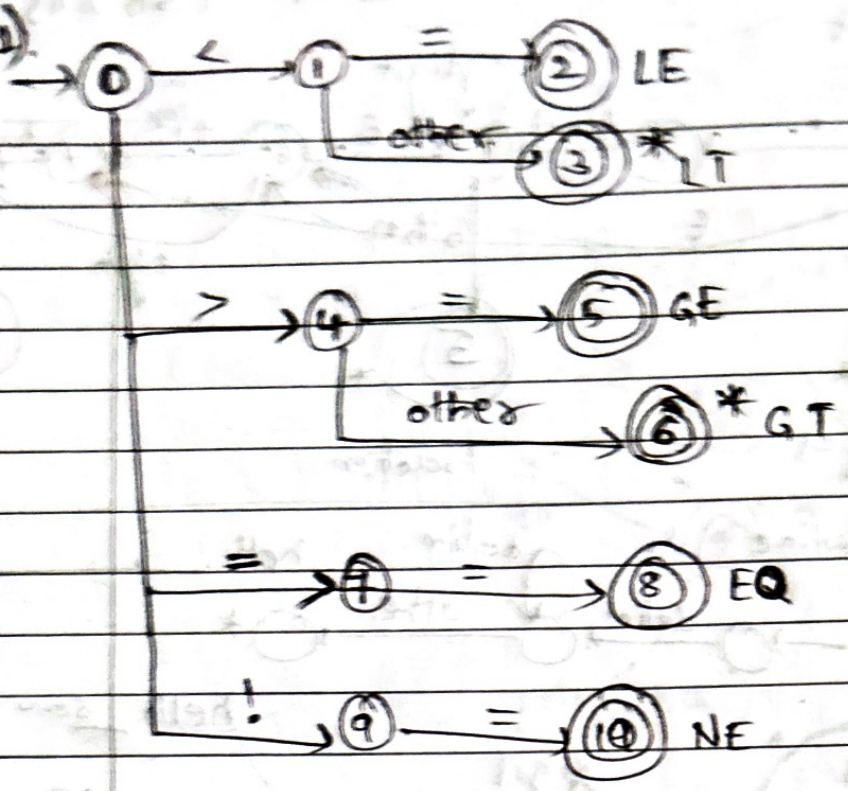
136.245E+24 1362.45E+24







1)




```
State=0;
for(;;)
{
  switch (state)
  {
    case 0:  ch = getchar();
             if (ch=='<')  state=1;
             else if (ch=='>') state=4;
             else if (ch=='=') state=7;
             else if (ch=='!') state=9;
             else  state 11;
             break;
    case 1;  ch = getchar();
             if (ch=='=')  state=2;
             else state=3;
             break;
    case 2 : return LE;
    case 3:  retract();
             return LT;
    case 4 : ch= getchar();
             if (ch=='=')  state=5;
             else state=6;
             break;
```

```
case 5: return GE;
case 6: retract();
        return GT;
case 7: ch = getchar();
        if (ch == '=') state = 8;
        else state = 11;
        break;
case 8: return EQ;
case 9: if (ch == '=') state = 10;
        else state = 11;
        break;
case 10: return NE;
case 11: /* Identify next token */
}
}
```

Module-2 & module-3 (choice)

*) CFG (context free grammer):

$S \rightarrow aS$

$G = (V, T, P, S)$

- V: Variables
- T: terminal
- P: production
- S: start symbol

*): Derivations: The process of obtaining string of terminals and/or non-terminals from the start symbol by applying some set of ~~options~~ productions is called "derivation".

$E \rightarrow E+E \mid E * E \mid (E) \mid id$

Parse derivations :

Exp :- $-(id+id)$

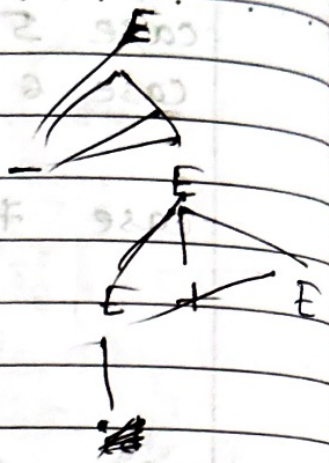
LMD

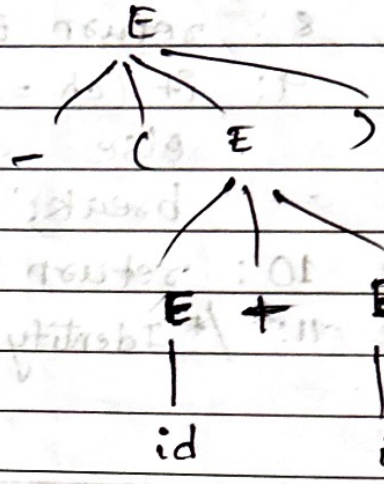
$E = -(E)$

$E = -(E+E)$

$E = -(id+E)$

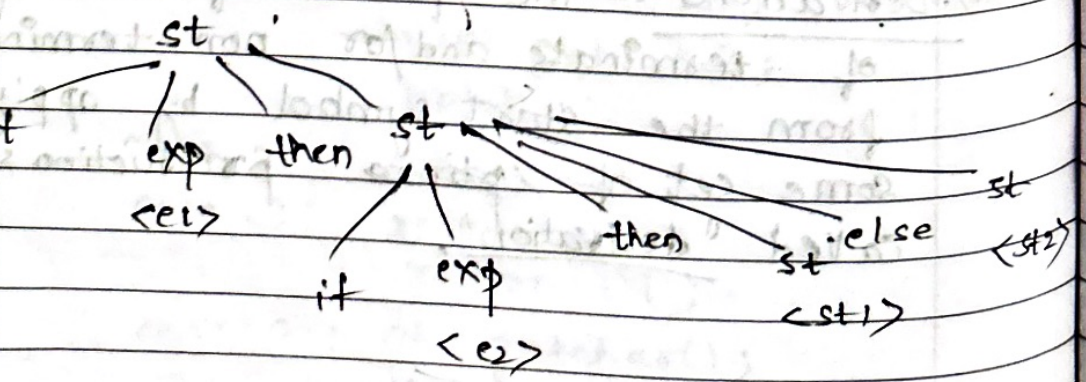
$E = -(id+id)$



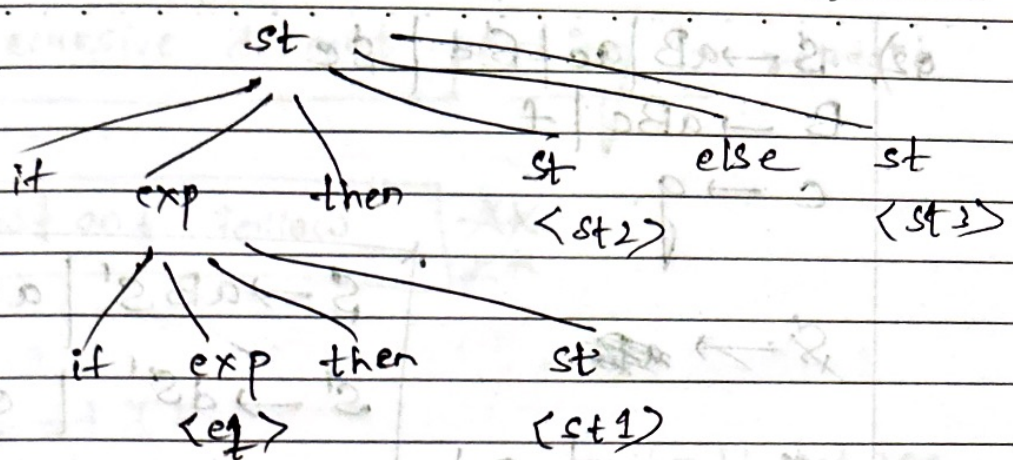


*). Ambiguous grammer : is the one that produces more than one Leftmost derivations or more than one rightmost derivation for the same tode.

a). St ⇒ if 'exp' then 'st'
if 'exp' then 'st' else 'st'
| other



Date __/__/__
Page _____



Elimination of left recursion

$A \rightarrow A\alpha \mid \beta$

Q) $E \rightarrow E+T \mid T$
$T \rightarrow T * F \mid F$
$F \rightarrow (E) \mid id$

$E \rightarrow TE'$
$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$
$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid id$

Eq: $A \rightarrow A\alpha \mid \beta$

$A \rightarrow \beta A'$
$A' \rightarrow \alpha A' \mid \epsilon$

$A \to A\alpha \mid \beta$

$A \to \beta A'$
$A' \Rightarrow \alpha A' \mid \epsilon$

Aα Aα

Q2) $S \to aB \mid ac \mid Sd \mid Se$
$B \to aBc \mid f$
$c \to g$

$S \to$ ~~[illegible]~~

~~$S \to Sd \mid aB'$~~

$S \to aBS' \mid acS'$
$S' \to dS' \mid eS' \mid \epsilon$
$B \to aBc \mid f$
$c \to g$

** left factoring :

rule: if $A \to \alpha\beta_1 \mid \alpha\beta_2$

$A \to \alpha A'$
$A' \to \beta_1 \mid \beta_2$

Eg: 1) $S \to iEtS \mid iEtSeS \mid a$
$S' \to E \to b$

Soln: $S \to itSS' \mid a$
$S' \to e \mid eS$

2) $S \to Ss+ \mid Ss* \mid a$

Soln: $S \to SsS' \mid a$
$S' \to + \mid *$

A B C → non-terminal

(a, bc → terminal Date __/__/__

Page _____

Recursive descent parsing :- retract:

First and Follow

1). $E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid id$

{ should get non-terminal } until move on

Soln: First (E) = { (, id }

First (E') = { +, ∈ }

First (T) = { (, id }

First (T') = { *, ∈ }

First (F) = { (, id }

[Following non terminal → First (non-ter

Follow (E) =

Follow (E') =

Follow (T) =

Follow (T') =

Follow (F) =

[Following terminal → follow (non-term

[If there is nothing go back to the production]

Follow (E) ⇒ [{ $,) }]

Follow (E') ⇒ { Follow (E) }

⇒ [{ $,) }]

$F \rightarrow (E) \mid id$

$E \rightarrow TE'$ ∴

Follow (T) = { first (E') } = {+, ε}

follow (E)

→ { +, $,) }

Follow (T') ⇒ { Follow (T) } → { +, $,) }

Follow (F) ⇒ { First (T') } ⇒ {*, ε}

follow (T')

→ { *, +, $,) }

2). D → L; T

L → L; id | id

T → int | real

L → idL'

L' → ;idL' | ε

sol^n:

First (D) ⇒ { id }

First (L) ⇒ { id }

First (T) ⇒ { int, real }

First (L') ⇒ {;, ε}

Follow (D) ⇒ { $ }

Follow (L) ⇒ { $; }

Follow (T) ⇒ { Follow (D) } ⇒ { $ }

Follow (L') ⇒ {;}

3). E → E + T | T

T → T F | F

F → F * | a | b

left recursion

A → Aα | β

A → βA'

A' → αA' | ε

sol^n:

E → T E'

E' → + T E' | ε

T → F T'

T' → F T' | ε

F → F * | a | b

F → a F' | b F'

F' → * F' | ε

DF:

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow FT' \mid \epsilon$

$F \rightarrow a\,F' \mid b\,F'$

$F' \rightarrow *F' \mid \epsilon$

sol":

First (E) = {a, b}

First (E') = {+, ε}

First (T) = {a, b}

First (T') = {ε, a, b}

First (F) = {a, b}

First (F') = {*, ε}

Follow (E) = {$}

follow (E') = Follow (E) = {$}

follow (T) = ~~Follow (~~ First (E') = {+, ε} follow(E')

~~Follow (T') = Follow (T) = {+, ε}~~

~~follow (F) = First (T') = {ε, a, b}~~ ~~follows(T')~~

Follow (T): ~~{+.~~ = {+, $}

Follow (T') = Follow (T) = {+, $}

Follow (F) = First (T') = {ε, a, b} ~~follow (T')~~

= {+, $, a, b}

Follow (F') = Follow (F) = {a, b, +, $}

Date __/__/__
Page

4) $D \to L;T$
$L \to L;id \mid id$
$T \to int \mid real$

$A \to A\alpha \mid \beta$

$A \to \beta A'$

$A' \to \alpha A' \mid \epsilon$

Soln:
$D \to L;T$
$L \to idL'$
$L' \to ;idL' \mid \epsilon$
$T \to int \mid real$

Q) $S \to T\,S \mid [\,S\,]\,S \mid \geq S \mid \epsilon$
$T \to (\alpha)$
$X \to T\,X \mid [\,X\,]\,X \mid \epsilon$

Soln:
First(S) = { [,), ϵ, (}
First(T) = { (}
First(X) = { [, ϵ, (}

consider only two
$T\,S$
$T\,X$

Follow(S) = { $,] }

follow(T) = { first(S) } ⇒ { [,), (, ϵ } follow(S)

{ [,), (, $,] }

∪

{ first(X) } ⇒ { [, ϵ, (} follow(X)

{ [, (,] }

follow(T) ⇒ { [,), (, $,] }

follow(X) = {] }

Q) [For the given grammer, construct predictive parsing table].

Q).

Q). $E \longrightarrow E+T \mid T$

$T \longrightarrow T*F \mid F$

$F \longrightarrow (E) \mid id$

Step1) left recursion:

$E \longrightarrow TE'$

$E' \longrightarrow +TE' \mid \epsilon$

$T \longrightarrow FT'$

$F' \longrightarrow *FT' \mid \epsilon$

$F \longrightarrow (E) \mid id$

$fi(E) = \{(, id\}$

$fi(T) = \{(, id\}$

$fi(F) = \{(, id\}$

$fi(E') = \{+, \epsilon\}$

$fi(T') = \{*, \epsilon\}$

$fol(E) = \{\$,)\}$

$fol(T) = \{+,), \$\}$

$fol(F) = \{+, *,), \$\}$

$fol(E') = \{\$,)\}$

$fol(T') = \{+,), \$\}$

if ϵ, then follow()

Input symbol

Non-terminals	(	id	+	*	$	)
E	$E \to TE'$	$E \to TE'$				
E'			$E' \to +TE'$		$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$	$T \to FT'$				
T'			$T' \to \epsilon$	$T' \to *FT'$	$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to (E)$	$F \to id$				

Q) [For the given grammer, construct predictive parsing table].

Q).

Q1). $E \rightarrow E+T \mid T$

$T \rightarrow T*F \mid F$

$F \rightarrow (E) \mid id$

Step1) left recursion:

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid id$

$fi(E) = \{(, id\}$

$fi(T) = \{(, id\}$

$fi(F) = \{(, id\}$

$fi(E') = \{+, \epsilon\}$

$fi(T') = \{*, \epsilon\}$

$fol(E) = \{\$,)\}$

$fol(T) = \{+,), \$\}$

$fol(F) = \{+, *,), \$\}$

$fol(E') = \{\$,)\}$

$fol(T') = \{+,), \$\}$

if ϵ, then follow()

Input symbol

Non-terminals	(	id	+	*	$	)
E	$E \rightarrow TE'$	$E \rightarrow TE'$				
E'			$E' \rightarrow +TE'$		$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$	$T \rightarrow FT'$				
T'			$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$	$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow (E)$	$F \rightarrow id$				

id + id * id

Date / /
Page

non-recursive predicting parsing

Method	Stack	Input	Action
	E $	id + id * id	
	T E' $	id + id * id	E → T E'
	F T' E' $	id + id * id	E → F T'
id	~~id~~ T' E' $	~~id~~ + id * id	E → id
id	T' E' $	+ id * id	
id	~~ϵ~~ E' $	+ id * id	T' → ϵ
id +	~~+~~ T E' $	~~+~~ id * id	
	F T' E' $	id * id	E' → + T E'
id + id	~~id~~ T' E' $	id * id	T → F T'
			F → id
id + id *	~~*~~ F T' E' $	~~*~~ id	T' → * F T'
id + id * id	~~id~~ T E' $	id	F → id
	T' E' $		
id + id * id	ϵ E' $		T' → ϵ
id + id * id	ϵ $		E' → ϵ

2). $S \longrightarrow A$

Input : id * id

$S \longrightarrow (S * A)$

$A \longrightarrow id$

First (S) $\Rightarrow$

*) For the given grammer find right ~~sequential~~ Sentential

(3). $S \longrightarrow 0S1 \mid 01$ $\longrightarrow$ right sequential

0 0 0 0 1 1 1 1

Right Sentential form	Handles	Reducing production
0 0 0 0 1 1 1 1	0 1	$S \rightarrow 01$
0 0 0 S 1 1 1	0 S 1	$S \rightarrow 0S1$
0 0 S 1 1	0 S 1	$S \rightarrow 0S1$
0 S 1	0 S 1	$S \rightarrow 0S1$
S		

Q2). "ss+a*a+" $S \rightarrow SS+ \mid SS* \mid a$

soln:

Right sentential form	Handles	Reducing production
~~ss~~ +a*a+	ss+	$S \rightarrow SS+$
Sa*a+	a	$S \rightarrow a$
~~SS~~*a+	ss*	$S \rightarrow SS*$
Sa+	a	$S \rightarrow a$
~~S,S~~+	SS+	$S \rightarrow SS+$
S		

* only For top-down parsing left recursion should be done

Bottom-up parsing

Shift-reduce parsing

Q). $E \rightarrow E+T \mid T$

$T \rightarrow T*F \mid F$

$F \rightarrow (E) \mid id$

Date __/__/__
Page _____

STACK	Input	Action
$	$id_1 * id_2$	shift 'id'
$id1	$*id_2$	Reduce 'F → id'
$F	$*id_2$	Reduce 'T → F'
$T	$*id_2$	shift '*'
$T*	id_2	shift 'id'
$T*id2	$	Reduce 'F → id'
$T*F	$	Reduce 'T → T*F'
$T	$	Reduce 'E → T'
$E	$	Accept

9). S → 0S0 | 1S1 | 2

Input 10201

Stack	Input	Action
$	10201$	shift 1
$1	0201$	shift '0'
$10	201$	shift '2'
$102	01$	S → 2
$10S	01$	shift '01'
$10S0	1$	S → 0S0
$1S	1$	shift '1'
$1S1	$	S → 1S1
$S	$	ACCEPT

Syntax Analysis II (mod-3)

Date __/__/__
Page

Q) construct a single LR parser for a given grammar

a) $E \to BB$ rule 1 ($r1$)
$B \to cB$ rule 2 ($r2$)
$B \to d$ rule 3 ($r3$)

a) construct LR(0) automaton
b) Build SLR Parsing table.
c) parse the given string : ccdd$

Sol^n): Step 1 :- Augmented grammar

$E' \to E$
$E \to BB$
$B \to cB$
$B \to d$

Step 2 : closure Items.

$E' \to .E$
$E \to .BB$
$B \to .cB$
$B \to .d$

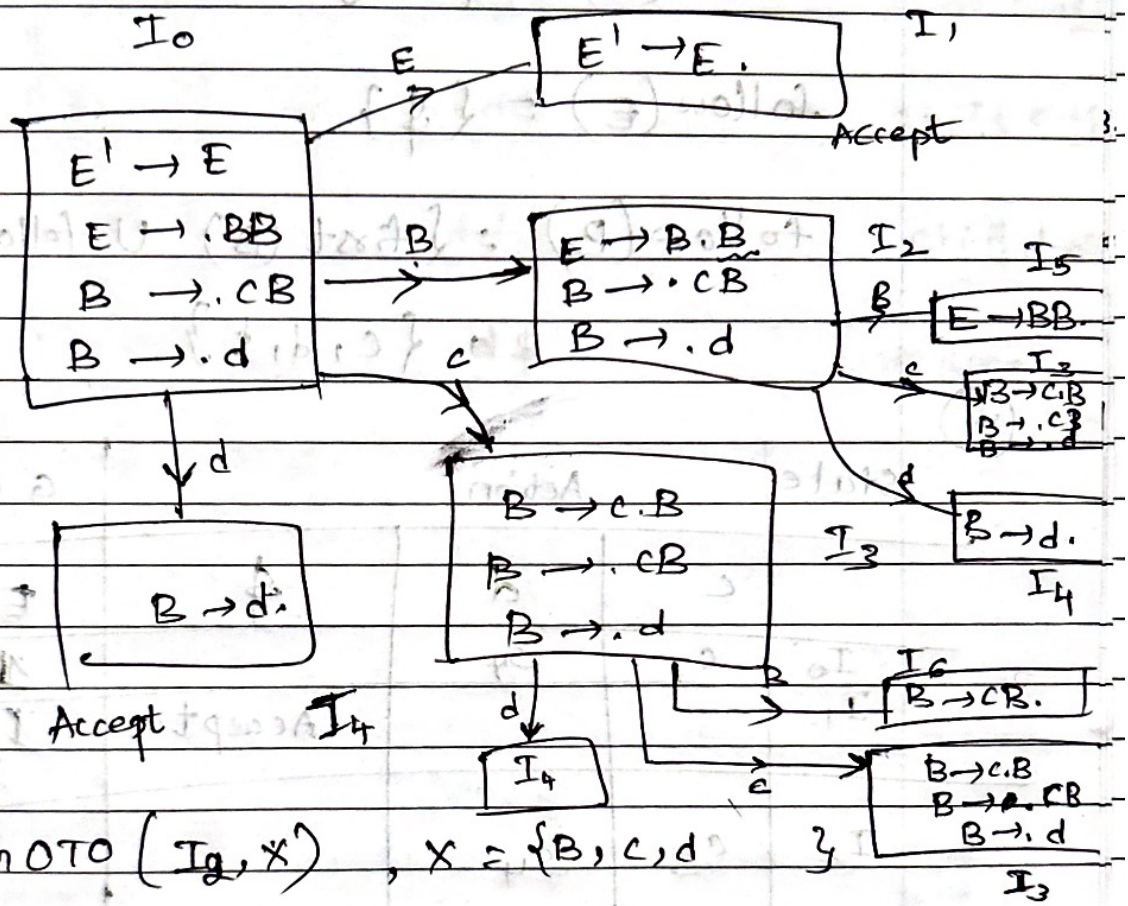
Step 3 : GOTO Function GOTO (I, X)

<after . first letter> where $X = \{E, B, c, d\}$

Step 4 :- Automaton $\Rightarrow$ LR(0)

* after • terminal neglect
• non-terminal write production

step 4:-



GOTO (I_0, X) , $X = \{B, C, d\}$

GOTO (I_3, X) , $X = \{B, C, d\}$

here I_3 is looping, leave as it is.

*) step5 :-

NOTE: If End of production see the follow of Production

If follow (Production) has {c, d, $} then check rule

NOTE ($E \to \underline{B}\underline{B}$) first(B) U follow

Date / / Page

first(E) = {c, d}

first (B) = {c, d}

follow (E) = {$}

follow (B) = {first (B) U follow (E)}

= {c, d, $}

(b)

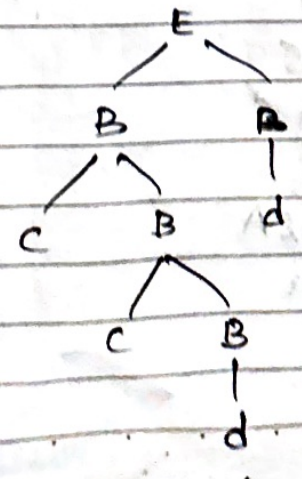
state	Action			GOTO	
	c	d	$	E	B
I_0	S_3	S_4		1	2
I_1			Accept		
I_2	S_3	S_4			5
I_3	S_3	S_4			6
I_4	r_3	r_3	r_3		
I_5			r_1		
I_6	r_2	r_2	r_2		

(4) (c). Parse the given string ccdd$ Date __/__/__ Page _____

Stack	Input	Action
$0	ccdd$	shift c→s3
$0c3	cdd$	shift c→s3
$0c3c3	dd$	shift d→s4
$0c3c3d4	d$	reduce B→d
$0c3c3BG	d$	reduce B→cB
$0c3cBG	d$	reduce B→cB
$0BG2	d$	shift d→s4
$0B2d4	$	reduce B→d
$0B2B5	$	reduce E→BB
$0E1	$	Accept

B+3·6

tree:



H.W

2). $r1 : S \to cAd$

$r2 : A \to ab$

$r3 : A \to e$

string : ced $

H.W

3). $S \to DD$

$D \to aD$

$D \to d$

string aadd $.

2) Sol^n : Step1 : Augmented grammer :

Q). consider the grammer :

$S \to AS \mid b$

$A \to SA \mid a$

$S \to AS$ r1

$S \to b$ r2

$A \to SA$ r3

$A \to a$ r4

*). construct LR(0) automation.

*). parse the given string 'abab'

Sol^n : Step1 : Augmented grammer.

$S' \to S$

$S \to AS$

$S \to b$

$A \to SA$

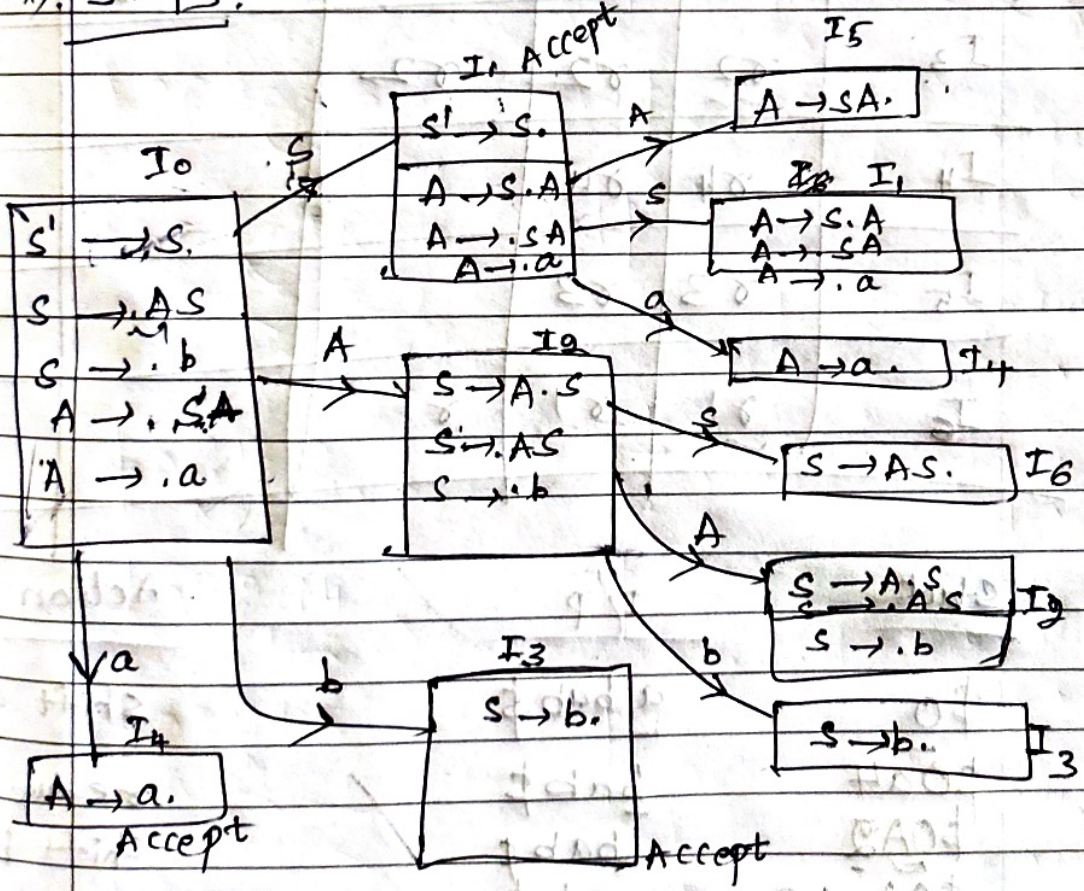
$A \to a$

Date __/__/__
Page _____

*) Step2 :- closure items :-

$S' \rightarrow .S$
$S \rightarrow .AS$
$S \rightarrow .b$
$A \rightarrow .SA$
$A \rightarrow .a$

*) Step3 :-



step4 :- First(S) = {a, b}

First (A) = {a, b}

Follow (S) = ~~First(A)~~ ∪ ~~follow(S)~~ = {$, a, b}

Follow (A) = {a, b}

NOTE: $I_4 \rightarrow (A \rightarrow a.)$ ∴ follow(A) ⇒ {a, b} ⇒ r_4^a, r_4^b

Date / /
Page

State	Action			Goto	
	a	b	$	S	A
I_0	s4	s3		1	2
I_1	s4		Accept	1	5
I_2		s3		6	2
I_3	r2	r2	r2		
I_4	r4	r4			
I_5	r3	r3			
I_6	r1	r1	r1		

Stack	I/P	Action
$0	abab$	shift a4
$0a4	bab$	reduce A→a
$0A2	bab$	shift b3
$0A2b3	ab$	reduce S→
$0A2S6	ab$	reduce S→
$0A2S1	ab$	shift a4
$0S1a4	b$	reduce A→a
$0S1A5	b$	reduce A→
$0A2	b$	shift b3
$0A2b3	$	reduce S→
$0A2S6	$	reduce S→
$0S1	$	Accept